

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

ФГБОУ ВО «Донской государственный технический университет»

Кафедра «Кибербезопасность информационных систем»

Лабораторная работа № 1

«Создание Web-приложения с использованием фреймворка Flask для
расчёта теоретической температуры горения»

Методические указания к выполнению
лабораторной работы по дисциплине
«Разработка защищённых Web-ресурсов»

Форма обучения: очная / очно-заочная

г. Ростов-на-Дону 2026 г.

Примечание: Приложение, которое вы начнёте разрабатывать, будет использоваться в последующих лабораторных работах. В лабораторной работе № 3 вы построите для него модель угроз, далее последовательно добавите аутентификацию, разграничение доступа, шифрование данных и защиту от типовых атак.

1. Цель и задачи

Цель работы: создание Web-приложения для технологических расчётов доменной плавки с использованием web-фреймворка Flask.

В лабораторной работе №1 создаётся проект, устанавливается фреймворк Flask, разрабатывается домашняя страница приложения и первый эндпоинт контроллера. По завершении части приложение запускается и отдаёт браузеру готовую страницу.

2. Подготовка проекта и среды разработки

2.1. Создание проекта и виртуального окружения

Создайте проект с произвольным названием.

Создайте виртуальное окружение для проекта, которое назовите `venv`. Для этого выполните команду:

```
python -m venv venv
```

Для активации виртуального окружения выполните команду:

```
source venv/bin/activate      # Linux / macOS  
venv\Scripts\activate        # Windows
```

В случае использования **PyCharm IDE** данные команды выполнять *не требуется*, поскольку виртуальное окружение будет создано и активировано автоматически. Обратите внимание: PyCharm по умолчанию присваивает окружению имя `.venv` (с точкой в начале) - именно так оно и отображено на рисунке 1.

Зачем нужно виртуальное окружение. Без него пакеты устанавливаются глобально, и два проекта, которым требуются разные версии одной библиотеки, начинают конфликтовать. Виртуальное окружение изолирует зависимости конкретного проекта.

2.2. Установка Flask и фиксация зависимостей

В проекте создайте файл Python и назовите его `app.py`.

Для начала работы с Flask в терминале выполните команду:

```
pip install flask
```

После чего установщик пакетов должен установить фреймворк.

Выполните команду:

```
pip freeze > requirements.txt
```

для сохранения зависимостей проекта.

Зачем фиксировать зависимости. Файл `requirements.txt` содержит перечень пакетов с указанием версий. По нему проект воспроизводится на другой машине одной командой `pip install -r requirements.txt`. В дальнейшем этот файл станет объектом анализа состава программного обеспечения.

3. Структура проекта

В данной лабораторной работе необходимо создать домашнюю страницу приложения для технологических расчётов и включить её в проект.

Все шаблоны (HTML-страницы), которые будет возвращать сервер по соответствующим запросам, должны находиться в каталоге `templates`. Создайте его в проекте, как показано на рисунке 1. В `templates` создайте два шаблона — `index.html` для домашней страницы и `tempr.html` для страницы расчёта температуры.

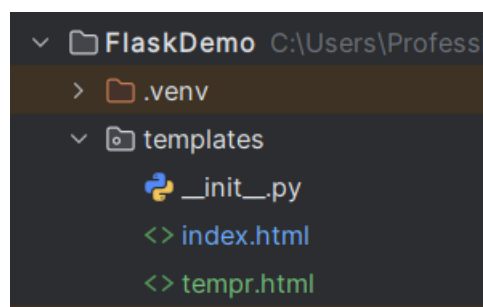


Рисунок 1 - Структура проекта на начальном этапе

Второй шаблон, `tempr.html`, вы наполните содержимым во второй части работы. Пока достаточно создать пустой файл: ссылка на него появится в навигационном меню домашней страницы.

Примечание о каталоге `templates`. Flask отыскивает шаблоны в этом каталоге по имени, поэтому файл `__init__.py` в нём не обязателен — `templates` не является пакетом Python. Во второй части работы вы создадите каталог `services`, который, в отличие от `templates`, будет именно пакетом: из него выполняется импорт, и `__init__.py` там необходим.

4. Разработка домашней страницы

В HTML-файле с названием `index.html` создайте разметку, пример реализации которой приведён на рисунке 2. В копирайте укажите год и Ваши имя и фамилию.

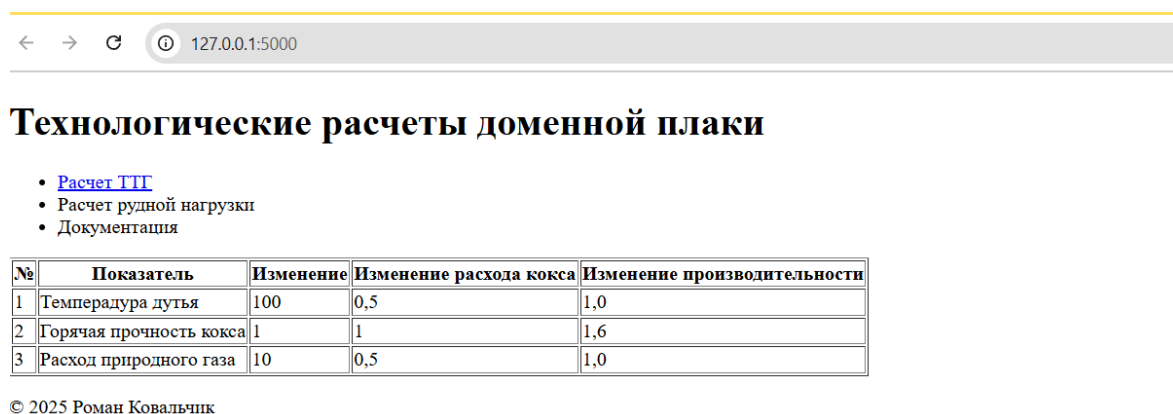


Рисунок 2 - Домашняя страница приложения (пример реализации)

4.1. Требования к разметке

С целью соблюдения правил семантической вёрстки:

- заголовок приложения и навигационное меню должны находиться внутри тега `<header>`, а само навигационное меню — внутри тега `<nav>`;
- основной контент должен быть реализован внутри тега `<main>`;
- подвальная часть страницы с Вашим именем и знаком копирайта — внутри тега `<footer>`.

Каркас разметки, который вам предстоит наполнить содержимым:

```

<!DOCTYPE html>
<html lang="ru">
<head>
  <meta charset="UTF-8">
  <title>Технологические расчёты доменной плавки</title>
</head>
<body>
  <header>
    <h1>Технологические расчёты доменной плавки</h1>
    <nav>
      <ul>
        <li><a href="/tempr">Расчёт ТТГ</a></li>
        <li>Расчёт рудной нагрузки</li>
        <li>Документация</li>
      </ul>
    </nav>
  </header>

  <main>
    <!-- Таблица влияния параметров дутья на показатели плавки -->
  </main>

  <footer>
    <p>&copy; 2025 Имя Фамилия</p>
  </footer>
</body>
</html>

```

Таблицу основного содержимого составьте по рисунку 2. Она носит справочный характер и показывает, как изменение параметров дутья влияет на расход кокса и производительность печи.

4.2. Дальнейшее развитие страницы

В дальнейшем к страницам будут применены стили CSS. Также в домашнюю страницу может быть добавлено изображение при помощи соответствующего тега по Вашему желанию.

5. Реализация backend-части приложения

Контроллер, состоящий из набора эндпоинтов, реализуйте в файле `app.py`.

Создайте первый эндпоинт, который при запуске приложения должен возвращать на фронтенд шаблон созданной Вами домашней страницы приложения (рисунок 3).

```

from flask import Flask, request
from flask import render_template

app = Flask(__name__)

@app.route(rule: '/', methods=['GET']) new *
def index():
    return render_template('index.html')

```

Рисунок 3 - Создание эндпоинта домашней страницы

Предварительно импортируйте класс `Flask`, а затем создайте его экземпляр с названием `app`, как показано на рисунке 3. Импортируйте `request` - он понадобится во второй части работы, когда потребуется получить данные из формы, - а также `render_template` для отправки шаблонов. Функция `render_template` принимает в качестве аргумента HTML-файл, который будет отправлен при переходе в данный эндпоинт.

Как это работает. Декоратор `@app.route` связывает URL-путь с функцией-обработчиком: при обращении по адресу `/` Flask вызовет функцию `index()`. Параметр `methods=['GET']` указывает, какие методы HTTP допустимы для этого пути.

Затем добавьте следующий код для запуска приложения в режиме отладки (рисунок 4).

```

from flask import Flask, request
from flask import render_template

app = Flask(__name__)

@app.route(rule: '/', methods=['GET']) new *
def index():
    return render_template('index.html')

if __name__ == '__main__':
    app.run(debug=True)

```

Рисунок 4 - Запуск приложения в режиме отладки

Конструкция `if __name__ == '__main__':` обеспечивает запуск сервера только при непосредственном выполнении файла, а не при его импорте. Параметр `debug=True` включает режим отладки: приложение автоматически перезагружается при изменении кода, а при возникновении ошибки возвращает подробную диагностическую страницу.

6. Запуск приложения и проверка

Осуществите запуск приложения путём выполнения команды в терминале:

```
python app.py
```

В случае отсутствия ошибок должно появиться следующее сообщение в терминале (рисунок 5).



```
PS C:\Users\Professional\Desktop\HackRF\FlaskDemo> python app.py
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 883-211-756
127.0.0.1 - - [21/Sep/2025 08:57:31] "GET / HTTP/1.1" 200 -
```

Рисунок 5 - Сообщение об успешном запуске приложения

Данное сообщение указывает, что веб-приложение запущено на локальном хосте.

При переходе по URL `http://127.0.0.1:5000` в браузере должна быть отражена домашняя страница веб-приложения, как показано на рисунке 2.

Остановка сервера осуществляется сочетанием клавиш `CTRL + C` из терминала.

Обратите внимание на предупреждение в выводе терминала: встроенный сервер Flask предназначен только для разработки и не должен использоваться в промышленной эксплуатации. Строки `Debug mode: on` и `Debugger PIN` также относятся к режиму разработки. К вопросу о том, что именно они означают с точки зрения защищённости, вы вернётесь в лабораторной работе № 2.

6.1. Если приложение не запускается

Три наиболее частые причины:

Сообщение	Причина и решение
<code>ModuleNotFoundError: No module named 'services'</code>	Запуск выполнен не из корневого каталога проекта. Перейдите в каталог, где лежит <code>app.py</code> , и запустите оттуда
<code>TemplateNotFound: index.html</code>	Шаблон лежит не в каталоге <code>templates</code> либо имя файла набрано с другим регистром
<code>Address already in use</code> (порт 5000)	Порт занят другой программой. На macOS его использует служба AirPlay Receiver: отключите её в системных настройках либо запустите приложение на другом порту — <code>app.run(debug=True, port=5001)</code>

7. Фиксация результата

```
git init
git add .
git commit -m "LP1: расчётчик теоретической температуры горения"
```

8. Контрольные вопросы

1. Что такое виртуальное окружение и какую проблему оно решает?
Зачем нужен файл `requirements.txt`?
2. Чем каталог `templates` отличается от пакета `services`? Почему в одном нужен `__init__.py`, а в другом нет?
3. Зачем бизнес-логика вынесена в отдельный пакет? Приведите три довода.
4. Как связываются URL-путь и функция-обработчик в Flask? Что делает `render_template` и где она ищет файлы?
5. Что даёт параметр `debug=True`? Почему встроенный сервер не предназначен для промышленной эксплуатации?

9. Перечень ссылок

1. Документация Flask: <https://flask.palletsprojects.com/en/stable/>
2. Методические указания о начале работы и установке Flask:
<https://rutube.ru/video/5d5132472bd7dca9e828d1b77cbba284>
3. Jinja2 Template Designer Documentation: <https://jinja.palletsprojects.com>
4. Расчёт теоретической температуры горения в горне доменной печи при вдувании пылеугольного топлива // КиберЛенинка.
5. Гринберг М. Разработка веб-приложений с использованием Flask на языке Python. — М.: ДМК Пресс.